Cognizant GenC Interview

Complete Questions & Answers Guide

Prepared for: Vishal Deshmukh | Interview Date: 14 May 2026

SECTION 1: OOP (Object Oriented Programming)

Q1. Class ■■■■■■ ■■■? (What is a Class?)

Class ■■■■■■■ ■■ blueprint/■■■■■ ■■ object ■■■■■■■■■■■■ ■■■■■■■. ■■■■■■ variables ■■■ methods ■■■■■■.

```
class Student {
String name;
int age;
void study() {
System.out.println("Studying...");
}
}
```

Q2. Object ■■■■■■ ■■■? (What is an Object?)

Object ■■■■■■■ class ■■ actual instance — class ■■■■■■■ ■■■■■■■■ actual ■■■■■■.

```
Student s = new Student(); // Object ■■■■■
s.name = "Vishal";
s.age = 21;
s.study();
```

Q3. Inheritance ■■■■■■ ■■■? Types ■■■■■.

■■ class ■■■■■■■■ class ■■ properties ■■■■■. 3 types: Single, Multilevel, Hierarchical.

```
// Single Inheritance
class Animal { void eat() { System.out.println("Eating"); } }
class Dog extends Animal { void bark() { System.out.println("Barking"); } }

// Multilevel Inheritance
class A { } class B extends A { } class C extends B { }

// Hierarchical Inheritance
class Animal { void eat() { } }
class Dog extends Animal { }
class Cat extends Animal { }
```

Q4. Polymorphism ■■■■■■ ■■■? Types ■■■■■.

■■■■■ ■■■■■■■ function ■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■. 2 types: Compile time (Overloading), Runtime (Overriding).

```
// Method Overloading (Compile Time)
class Calculator {
int add(int a, int b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }
}

// Method Overriding (Runtime)
class Animal { void sound() { System.out.println("Some sound"); } }
class Dog extends Animal { void sound() { System.out.println("Woof"); } }
class Cat extends Animal { void sound() { System.out.println("Meow"); } }
```

Q5. Encapsulation ■■■■■■ ■■■?

Data ■■■■■■ ■■■■ getter/setter methods ■■■■■■■ access ■■■■■.

```
class BankAccount {
private double balance;
public double getBalance() { return balance; }
public void deposit(double amount) {
if(amount > 0) balance += amount;
}
}
```

Q6. Abstraction ■■■■■■■ ■■■■?

■■■■■ ■■■■■■■■ ■■■■■■■, ■■■■■ ■■■■■■. Abstract class ■■■■■■ Interface ■■■■■■■.

```
abstract class Shape {
abstract void draw();
}
class Circle extends Shape {
void draw() { System.out.println("Drawing Circle"); }
}
```

Q7. Interface ■■■■■■■ ■■■■?

Interface ■■■■■■■ 100% abstract class — ■■■■■ methods abstract ■■■■■■. Multiple implement ■■■■■ ■■■■■■.

```
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable {
public void fly() { System.out.println("Duck flying"); }
public void swim() { System.out.println("Duck swimming"); }
}
```

Q8. Abstract Class vs Interface ■■■■?

Abstract Class	Interface
■■■■■ methods implement ■■■■■■	■■■■■ methods abstract
extends ■■■■■■■	implements ■■■■■■■
■■■■ inherit ■■■■■ ■■■■■	Multiple implement ■■■■■ ■■■■■■
Constructor ■■■■■	Constructor ■■■■■

Q9. Constructor ■■■■■■■ ■■■■?

Object ■■■■■■■ automatically call ■■■■■■■ special method. Class ■■■■■ ■■■■■■■ ■■■■■.

```
class Student {
String name;
int age;
Student(String name, int age) { // Constructor
this.name = name;
this.age = age;
}
}
Student s = new Student("Vishal", 21);
```

Q10. Static vs Non-Static ■■■■?

Static — ■■■■■ objects share ■■■■■■, class level. Non-Static — ■■■■■■■■■ object ■■ ■■■■■■■■.

```
class Example {
static int count = 0; // ■■■■■ objects share
int id; // ■■■■■■■■■ object ■■ ■■■■■■
static void showCount() {
System.out.println("Count: " + count);
}
}
```

```
Example.showCount(); // Object ■ ■■■■■ call
```

SECTION 2: Java

Q11. String Methods ■■■■■ ■■■■?

```
String s = "Vishal Deshmukh";
s.length() // 15
s.toUpperCase() // VISHAL DESHMUKH
s.toLowerCase() // vishal deshmukh
s.charAt(0) // 'V'
s.indexOf("Desh") // 7
s.substring(0, 6) // "Vishal"
s.replace("Vishal", "Raj") // "Raj Deshmukh"
s.trim() // spaces remove
s.contains("Desh") // true
s.split(" ") // ["Vishal", "Deshmukh"]
```

Q12. == vs .equals() ■■■?

== memory address compare ■■■■. .equals() actual value compare ■■■■.

```
String a = new String("Hello");
String b = new String("Hello");
System.out.println(a == b); // false (memory different)
System.out.println(a.equals(b)); // true (value same)
```

Q13. Exception Handling ■■■■■■ ■■■?

Program error ■■■■■■■ crash ■■■ ■■■■ ■■■■■■■ try-catch ■■■■■■.

```
try {
int x = 10 / 0;
int[] arr = new int[5];
arr[10] = 1;
} catch (ArithmeticException e) {
System.out.println("Math error: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Array error: " + e.getMessage());
} finally {
System.out.println("Always executes");
}
```

Q14. Array vs ArrayList ■■■?

Array	ArrayList
Fixed size	Dynamic size
Primitive types	Objects store ■■■■
Fast	■■■■ slow
int[] arr = new int[5]	ArrayList<Integer> list = new ArrayList<>()

Q15. HashMap ■■■■ ■■■■ ■■■■■■?

```
HashMap&lt;String, Integer&gt; map = new HashMap&lt;&gt;();
map.put("Vishal", 90);
map.put("Raj", 85);
System.out.println(map.get("Vishal")); // 90
System.out.println(map.containsKey("Raj")); // true
map.remove("Raj");
for(String key : map.keySet()) {
System.out.println(key + " = " + map.get(key));
}
```

SECTION 3: SQL

Q16. Basic SQL Queries

```
-- Table ■■■■■
CREATE TABLE students (
id INT PRIMARY KEY,
name VARCHAR(50),
age INT,
marks FLOAT
);
-- Data ■■■■■
INSERT INTO students VALUES (1, 'Vishal', 21, 85.5);
-- Data ■■■■
SELECT * FROM students;
SELECT name, marks FROM students WHERE marks &gt; 80;
-- Update
UPDATE students SET marks = 95 WHERE name = 'Vishal';
-- Delete
DELETE FROM students WHERE id = 2;
-- Sort
SELECT * FROM students ORDER BY marks DESC;
```

Q17. JOIN Queries

```
-- INNER JOIN
SELECT s.name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;

-- LEFT JOIN
SELECT s.name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

Q18. Aggregate Functions

```
SELECT COUNT(*) FROM students; -- ■■■■ rows?
SELECT AVG(marks) FROM students; -- Average marks
SELECT MAX(marks) FROM students; -- ■■■■■■■ ■■■■■
SELECT MIN(marks) FROM students; -- ■■■■■■■ ■■■
SELECT SUM(marks) FROM students; -- Total
-- Group BY
SELECT dept_id, AVG(marks)
FROM students
GROUP BY dept_id HAVING AVG(marks) &gt; 75;
```

Q19. Primary Key vs Foreign Key

Primary Key — Unique identifier, duplicate ■■■■. Foreign Key — ■■■■■■■■ table ■■ refer ■■■■.

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE students (
id INT PRIMARY KEY,
name VARCHAR(50),
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

SECTION 4: DSA (Data Structures)

Q20. Array Operations

```
int[] arr = {5, 3, 8, 1, 9, 2};
// Traverse
for(int i = 0; i &lt; arr.length; i++) {
System.out.print(arr[i] + " ");
}
// Maximum ■■■■■
int max = arr[0];
for(int i = 1; i &lt; arr.length; i++) {
if(arr[i] &gt; max) max = arr[i];
}
System.out.println("Max: " + max); // 9
```

Q21. Stack (LIFO — Last In First Out)

■■■■■■■: Books ■■ ■■■■. ■■■■■■ ■■■■■■■■ ■■■■■■ ■■■■■■.

```
Stack&lt;Integer&gt; stack = new Stack&lt;&gt;();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.peek()); // 30 (top)
System.out.println(stack.pop()); // 30 (remove)
System.out.println(stack.size()); // 2
```

Q22. Queue (FIFO — First In First Out)

■■■■■■■: ■■■■■/Line. ■■■■■■ ■■■■■■ ■■■■■■ ■■■■■.

```
Queue&lt;Integer&gt; queue = new LinkedList&lt;&gt;();
queue.add(10);
queue.add(20);
queue.add(30);
System.out.println(queue.peek()); // 10 (front)
System.out.println(queue.poll()); // 10 (remove)
System.out.println(queue.size()); // 2
```

Q23. Linked List

```
LinkedList&lt;String&gt; list = new LinkedList&lt;&gt;();
list.add("Vishal");
list.add("Raj");
list.addFirst("Amit");
list.addLast("Ravi");
System.out.println(list.getFirst()); // Amit
System.out.println(list.getLast()); // Ravi
list.remove("Raj");
```

SECTION 5: Coding Questions

Q24. Reverse a String

```
// Method 1 - StringBuilder
String s = "Vishal";
String reversed = new StringBuilder(s).reverse().toString();
System.out.println(reversed); // lahsiV

// Method 2 - Loop
String str = "Hello";
String rev = "";
for(int i = str.length()-1; i &gt;= 0; i--) {
rev += str.charAt(i);
```

```
}
System.out.println(rev); // olleH
```

Q25. Check Palindrome

```
String s = "madam";
String rev = new StringBuilder(s).reverse().toString();
if(s.equals(rev)) {
System.out.println("Palindrome ■■■");
} else {
System.out.println("Palindrome ■■■■");
}
```

Q26. Factorial

```
// Loop
int n = 5;
int factorial = 1;
for(int i = 1; i &lt;= n; i++) { factorial *= i; }
System.out.println("5! = " + factorial); // 120

// Recursion
int fact(int n) {
if(n == 0) return 1;
return n * fact(n-1);
}
```

Q27. Fibonacci Series

```
int n = 10;
int a = 0, b = 1;
System.out.print(a + " " + b + " ");
for(int i = 2; i &lt; n; i++) {
int c = a + b;
System.out.print(c + " ");
a = b;
b = c;
}
// Output: 0 1 1 2 3 5 8 13 21 34
```

Q28. Check Prime Number

```
int num = 17;
boolean isPrime = true;
if(num &lt; 2) isPrime = false;
for(int i = 2; i &lt;= Math.sqrt(num); i++) {
if(num % i == 0) { isPrime = false; break; }
}
if(isPrime) System.out.println(num + " Prime ■■■");
else System.out.println(num + " Prime ■■■■");
```

Q29. Bubble Sort

```
int[] arr = {64, 34, 25, 12, 22};
int n = arr.length;
for(int i = 0; i &lt; n-1; i++) {
for(int j = 0; j &lt; n-i-1; j++) {
if(arr[j] &gt; arr[j+1]) {
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
}
}
}
// Output: 12 22 25 34 64
```

Q30. Find Duplicates in Array

```
int[] arr = {1, 2, 3, 2, 4, 3, 5};
HashSet&lt;Integer&gt; set = new HashSet&lt;&gt;();
for(int num : arr) {
if(!set.add(num)) {
System.out.println("Duplicate: " + num);
}
}
// Output: Duplicate: 2, Duplicate: 3
```

Q31. Count Vowels in String

```
String s = "Vishal Deshmukh";
int count = 0;
for(char c : s.toLowerCase().toCharArray()) {
if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
}
System.out.println("Vowels: " + count); // 4
```

Q32. Swap Two Numbers Without Temp Variable

```
int a = 10, b = 20;
a = a + b; // 30
b = a - b; // 10
a = a - b; // 20
System.out.println("a=" + a + " b=" + b); // a=20 b=10
```

SECTION 6: HR Interview Questions

Q33. Tell me about yourself?

"My name is Vishal Deshmukh. I completed my degree in [Branch] from [College Name]. I have strong knowledge in Java, OOP, SQL and Data Structures. I worked on a project called [Project Name] where I built [brief description]. I am a hardworking, quick learner and a good team player. I am excited to start my career at Cognizant and contribute to the team."

Q34. Why do you want to join Cognizant?

"Cognizant is one of India's top IT companies with a great learning culture and innovative environment. The GenC program gives freshers like me an excellent opportunity to learn, grow, and work on real-world projects. I believe Cognizant is the best place to start and build my career."

Q35. What are your Strengths?

"I am a quick learner, hardworking, and a good team player. I can adapt to new technologies easily and I stay calm under pressure."

Q36. What are your Weaknesses?

"Sometimes I spend too much time perfecting small details. But I am working on managing my time better by setting clear deadlines for each task."

Q37. Where do you see yourself in 5 years?

"In 5 years, I see myself as a skilled software developer at Cognizant, leading projects, mentoring junior team members, and contributing to the company's growth."

Q38. Are you okay with relocation?

"Yes, absolutely. I am flexible and ready to work from any location as required by Cognizant."

Q39. Do you have any questions for us?

"Yes! What technologies will I be working with during my initial training period at Cognizant?"

SECTION 7: Last Minute Tips - ■■■■■■■■■

■■■	■■■ ■■■■■■
■■ ■■■■■■	OOP + Coding practice ■■■, ■■■ ■■■■
■■■■■ 11:00 AM	Laptop charge, internet check ■■■
■■■■■ 11:20 AM	app.joinsuperset.com ■■ login ■■■
■■■■■ 11:30 AM	Interviews tab ■■■■, interview ■■■■
Interview ■■■■■■■	Smile ■■■, ■■■ ■■■■, College ID ■■■■ ■■■■

■■■■, ■■■■■■ ■■■■■■ Select ■■■■■! All the Best! Best of Luck!